

Senet AI

Code Guide

Note : this is a AI generated code guide for users who do not understand code completely.

Overview — How the Files Fit Together

The project is split into 6 Python files. Each file has one clear job. They communicate with each other by calling functions, passing data back and forth like workers on an assembly line. Understanding what each file is responsible for makes the whole project much easier to follow.

Function / Part	What it does — plain English
<code>main.py</code>	The entry point — the first file that runs. Checks if AI models are trained, trains any missing ones, then opens the game window.
<code>game_engine.py</code>	The rule book — contains all the logic for the board, legal moves, special squares, and win detection. No AI or graphics here.
<code>agent.py</code>	The brain — stores the Q-table (the AI's learned knowledge) and decides which move to make based on that knowledge.
<code>training.py</code>	The coach — runs thousands of practice games between two agents and updates their knowledge after every single move.
<code>gui.py</code>	The visual layer — draws the board, pieces, and buttons using Pygame. Responds to your mouse clicks and runs the game loop.
<code>utils.py</code>	The toolbox — small helper functions shared across files: smoothing graphs, saving logs, creating folders, showing progress.

□ A good way to read this guide: start here for the big picture, then read each section in order. By the end you'll be able to open any file and understand exactly what every function does.

1. main.py — The Entry Point

main.py — Run this file to start everything
Role: project manager — coordinates training and launches the GUI

main.py is the first file that executes when you run the project. It acts as a coordinator: it figures out what needs to be done before the game can open, does it, and then hands control over to the GUI. You never need to edit this file — you only run it.

What Happens When You Run It

1. Python starts executing main.py from the top.
2. It checks the models/ folder for three files: agent_easy.pkl, agent_medium.pkl, agent_hard.pkl.
3. Any model file that is missing gets trained right then — easy takes ~30 seconds, hard ~10 minutes.
4. Once all three models exist, it imports SenetGUI from gui.py and calls gui.run().
5. The Pygame window opens and you are in control.

Command-Line Options

You can modify this behaviour with optional flags:

```
python main.py      # normal run - trains missing models, then opens GUI
python main.py --skip-train  # skips training entirely, opens GUI immediately
python main.py --train-only  # trains all three levels and exits without opening GUI
python main.py --level hard  # re-trains the hard model only, then opens GUI
```

□ If you already have all three .pkl files (e.g. you ran training separately), main.py detects this and skips training automatically. It will print 'All models already trained' and go straight to the GUI.

Why This File Exists

Separating startup logic into main.py is a common Python pattern. It keeps gui.py and training.py focused on their own jobs. It also means you can run training separately (python training.py) or launch the GUI without re-training (-skip-train) without changing any code.

2. game_engine.py — The Rule Book

game_engine.py — Board, legal moves, special squares, win detection

Role: referee — the single source of truth for all game rules

game_engine.py contains the SenetGame class. This file knows absolutely nothing about AI or graphics. Its entire job is to enforce the rules of Senet. Every other file that needs to know about the board state — the agent, the GUI, the training loop — asks this file. There is only one place where the rules live, which means if you ever need to fix or change a rule, there is only one place to look.

How the Board is Stored

The board is a Python list of exactly 30 integers, one for each square on the board:

```
self.board = [1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 0, 0, ...]
```

Each integer means one thing:

- 0 — the square is empty
- 1 — Player 1 (Blue / Human) has a piece here
- -1 — Player 2 (Red / AI) has a piece here

The board follows an S-shaped path across the three rows. Row 0 (squares 0–9) goes left to right. Row 1 (squares 10–19) goes right to left. Row 2 (squares 20–29) goes left to right again. A piece at position 9 moves to position 10, which appears on the opposite end of the second row — this is the S-turn. A piece that moves past position 29 exits the board entirely.

□ Using a flat list of 30 numbers is deliberately simple. It means the entire board state can be converted to a tuple in one line and used as a dictionary key in the Q-table — no complex encoding needed.

Special Squares

Square	Constant Name	Effect
15	SAFE_SQUARE	A piece here cannot be captured or swapped. The opponent must skip it.
20	EXTRA_TURN_SQ	Landing here grants the same player an immediate extra turn.
27	TRAP_SQUARE	Landing here sends the piece back to square 1. A big penalty.

Key Functions Explained

Function / Part	What it does — plain English
<code>reset()</code>	Clears the board to the starting position: pieces alternate P1/P2 on the first 10 squares. Returns the initial state tuple.
<code>roll_dice()</code>	Returns <code>random.randint(1, 5)</code> . This simulates throwing four binary throwing sticks used in ancient Senet. Each face shows one of two sides, giving values 1–5.
<code>get_valid_moves()</code>	Loops through every square. For each piece belonging to the current player it checks whether moving by <code>dice_roll</code> squares is legal: destination must be empty, off the board, or occupied by a single unprotected opponent piece.
<code>move_piece(pos)</code>	The main action function. Moves the piece at <code>pos</code> , applies all side effects (swap, trap, safe, extra turn), checks the win condition, switches the current player, and returns (<code>new_state</code> , <code>reward</code> , <code>done</code> , <code>info</code>).
<code>get_state()</code>	Returns <code>tuple(self.board)</code> — a frozen snapshot of the board. Tuples are hashable so they can be used as dictionary keys in the Q-table.
<code>clone()</code>	Makes an exact independent copy of the current game object. Used in some lookahead scenarios during training without affecting the live game.

The Return Value of `move_piece()`

`move_piece()` returns four things every time it is called. This is the standard interface for RL environments:

- `new_state` — the board as a tuple after the move.
- `reward` — a number telling the agent how good or bad that move was (see reward table in `agent.py` section).
- `done` — True if the game is over, False if it continues.
- `info` — a dictionary with extra details: whether the move was invalid, whether an extra turn was granted, and who won.

□ *This four-value return format (state, reward, done, info) is the standard interface used by OpenAI Gym and most RL frameworks. Learning it here prepares you for professional RL library usage.*

3. `agent.py` — The Brain

`agent.py` — Q-table storage, action selection, learning update, save/load

Role: the AI — learns from experience and makes decisions

`agent.py` contains the `QLearningAgent` class. This is the AI. It has no knowledge of game rules whatsoever — it just receives a board state and a list of valid moves, looks up values in its Q-table, and returns the move with the highest value. Over thousands of episodes those values become accurate enough to play strategically.

The Q-Table — The Agent's Memory

The Q-table is a Python `defaultdict` (a dictionary that returns 0 for missing keys):

```
self.q_table[ (board_state_tuple, piece_position) ] = value
```

The key is a (state, action) pair. State is the full board as a 30-integer tuple. Action is the board index of the piece to move (an integer 0–29). The value is a float representing how good that move is expected to be from that position — higher is better. A brand-new agent has all values at 0.0. After training, strategically important moves have noticeably higher values than poor ones.

□ *The Q-table can grow to millions of entries. The hard agent typically stores over 5 million (state, action) pairs. Each entry is the result of experience — the agent has literally been in that situation and learned from it.*

How the Agent Chooses a Move

During training the agent uses an epsilon-greedy strategy:

- With probability ϵ (epsilon) — pick a completely random valid move. This is exploration.
- With probability $1 - \epsilon$ — pick the valid move with the highest Q-table value. This is exploitation.

Epsilon starts at 1.0 (completely random) and decays after every episode. By the end of training it is at 0.05 (mostly exploiting, 5% random). This schedule ensures the agent explores widely early on and then commits to the best strategy it has found.

During actual gameplay (not training), `choose_action_greedy()` is used — always picks the best known move, no randomness.

□ *Exploration vs exploitation is a fundamental challenge in RL. Too much exploration = the agent never commits to good moves. Too little = it gets stuck in a local optimum and never discovers better strategies.*

The Learning Update (Bellman Equation)

After every single move the agent's Q-table is updated using this formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \times [r + \gamma \times \max_{a'} Q(s',a') - Q(s,a)]$$

Breaking this down piece by piece:

- $Q(s,a)$ — the current score for this (state, action) pair in the table.
- r — the reward just received from `game_engine.move_piece()`.
- $\max Q(s',a')$ — the highest Q-value available in the next state — the best the agent could do from here.
- γ (gamma) — the discount factor. Multiplied by the future value to make near-term rewards worth more than distant ones.
- α (alpha) — the learning rate. Controls how big a step toward the new estimate is taken. Too high = unstable. Too low = very slow learning.
- The bracket $[r + \gamma \times \max_{a'} Q(s',a') - Q(s,a)]$ is called the TD error (temporal difference error). It is the gap between what we expected and what we got.

□ *You can think of this update as: new score = old score + small_step × (what actually happened – what we expected). Over thousands of games the scores converge to accurate estimates.*

Reward Structure

Event	Reward Value	Why this value?
Win the game	+1.00	The strongest positive signal. Winning is always the ultimate goal.
Lose the game	-1.00	Equal and opposite to winning. Strongly discourages losing.
Valid move made	+0.05	Small encouragement for legal play. Keeps the agent active.
Invalid move tried	-0.10	Double the valid-move reward as a penalty. Discourages illegal attempts.
Per-step penalty	-0.01	A tiny cost per move. Encourages the agent to win efficiently, not drag games out.

Saving and Loading Models

The `save()` function uses Python's pickle module to serialise the entire Q-table dictionary to a .pkl file. The `load()` function reads it back. This is what allows the agent to be trained once and then used indefinitely — the knowledge persists between sessions. The three model files are:

```
models/agent_easy.pkl    ← 2,000 episode Q-table    (~1.3M entries)
models/agent_medium.pkl  ← 10,000 episode Q-table   (~5.6M entries)
models/agent_hard.pkl    ← 50,000 episode Q-table   (~5.6M+ entries)
```

Difficulty Configurations

The static method `difficulty_config()` returns different hyperparameters per level:

Setting	Easy	Hard
Episodes	2,000	50,000
Alpha (α)	0.15	0.05
Gamma (γ)	0.90	0.99
Epsilon decay	0.990	0.9995

□ *Hard uses a lower alpha (more conservative updates) and higher gamma (values the future more) because it has many more episodes to converge slowly and carefully. Easy uses aggressive settings because it needs to learn something useful in only 2,000 games.*

4. training.py — The Coach

training.py — Self-play training loop, progress tracking, graph generation, model saving
Role: runs the learning process from start to finish

`training.py` is where learning actually happens. It creates two `QLearningAgent` instances, sets up a `SenetGame`, and runs them through thousands of complete games against each other. After every move it calls `agent.update()` to improve the Q-table. After every episode it decays epsilon and records the result. When training ends it saves the Q-table and generates graphs.

The Training Loop — Step by Step

- Two agents are created: `agent1` (plays as `PLAYER1`) and `agent2` (plays as `PLAYER2`).
- A new game is started: `game.reset()` — fresh board, all pieces in starting positions.
- The current player's agent rolls the dice: `game.roll_dice()`.
- `get_valid_moves()` returns the list of legal moves for this dice roll.
- If there are no valid moves the turn is passed to the opponent.
- The agent calls `choose_action(state, valid_moves)` — random or greedy depending on ϵ .
- `game.move_piece(action)` is called. It returns (`new_state`, `reward`, `done`, `info`).
- `agent.update(state, action, reward, new_state, next_valid_moves, done)` updates the Q-table.
- If `done` is `True` the episode is over. Win/loss is recorded and ϵ is decayed for both agents.
- Otherwise the loop continues with the next player's turn.
- This repeats for all episodes. Results are logged and saved.

The Step Limit

A maximum of 500 moves per episode is enforced. If neither player has won after 500 moves the game ends as a draw. This prevents pathological cases where two random agents loop indefinitely in the early stages of training when epsilon is near 1.0.

What Gets Saved After Training

- `models/agent_<level>.pkl` — the full Q-table and metadata, used by the GUI at game time.
- `graphs/training_<level>.png` — two-panel chart: win rate over episodes and average reward over episodes.
- `logs/log_<level>.csv` — raw per-episode win/reward data for further analysis if needed.

Running Training Manually

```
| python training.py --level easy # ~30 seconds
```

```
python training.py --level medium    # ~2 minutes
python training.py --level hard      # ~10 minutes
python training.py --level all       # trains all three in sequence
```

□ *Self-play means both agents improve at the same rate. Because the competition always matches the agent's current skill level, the opponent is never too easy (boring) or too hard (impossible). This creates a naturally calibrated learning environment.*

5. gui.py — The Visual Layer

gui.py — Pygame window, board rendering, click handling, How to Play guide
Role: everything the user sees and interacts with

gui.py contains the SenetGUI class. It uses the Pygame library to create a window, draw the game state 30 times per second, and respond to mouse clicks. It never changes the game state directly — it always goes through game_engine.move_piece() to do that, keeping the rules strictly in one place.

The Main Loop

The run() method loops forever. Every iteration (every frame at 30 FPS) it does three things in this order:

- 17._handle_events() — check for mouse clicks, button presses, or window close.
- 18._draw() — clear the screen and repaint: board, pieces, buttons, info panel, and guide overlay if active.
- 19.AI check — if it is the AI's turn and the game is not over, call _ai_move().

Drawing Functions

Function / Part	What it does — plain English
<code>_draw_board()</code>	Loops through all 30 squares and draws each one with the correct colour. Special squares get their own colour: green (Safe), salmon/orange (Trap), light blue (Extra Turn). Valid-move squares get a bright gold border.
<code>_draw_pieces()</code>	Loops through the board list. For every non-zero cell it draws a coloured circle (blue for P1, red for P2). The currently selected piece gets a gold ring instead of white so you can see which piece is active.
<code>_draw_buttons()</code>	Draws the row of buttons at the bottom of the screen. The currently active mode (Human vs AI, AI vs AI, or Guide) and the current difficulty level glow gold so you always know your current settings.
<code>_draw_info_panel()</code>	Draws the panel below the buttons showing the current status message, mode, difficulty, last dice roll, and the colour legend for the three special squares.
<code>_draw_guide_overlay()</code>	When the How to Play guide is active, this draws a semi-transparent dark overlay over the entire window, then renders the current tutorial slide on top with its title, body text, navigation arrows, and any square highlights.

Input Handling

Function / Part	What it does — plain English
<code>_handle_click(pos)</code>	Routes every mouse click: first checks if a guide navigation arrow was clicked, then checks buttons, then checks board cells. This routing keeps each handler small and focused.
<code>_handle_button(action)</code>	Receives an action string from the clicked button (e.g. 'new_game', 'hard', 'guide') and updates the game mode, difficulty, or guide state accordingly.

Function / Part	What it does — plain English
<code>_handle_cell_click(idx)</code>	Two-stage click logic: the first click on your own highlighted piece selects it (gold ring appears). A second click on any highlighted destination moves it. Clicking elsewhere deselects.
<code>_handle_guide_click(pos)</code>	Detects clicks on the Prev and Next arrow buttons inside the guide overlay and updates the guide_page counter.

The How to Play Guide

The guide is implemented as a list of 9 slide dictionaries defined at the top of `gui.py`. Each slide has a title, a list of text lines, and an optional list of board square indices to highlight. When guide mode is active, `_draw_guide_overlay()` reads the current slide, draws the content, and highlights the relevant squares on the board behind the overlay so players can see exactly which squares are being discussed.

❑ *The GUI loads all three .pkl model files at startup. If a model file is missing it shows a warning message in the status bar and falls back to random play for that difficulty level. You can still play — it just won't be as smart.*

Game Modes

Function / Part	What it does — plain English
Human vs AI	You control Blue (Player 1). Highlighted squares show legal moves. The AI controls Red and moves automatically with a 500ms pause so you can see what it does.
AI vs AI	Both players are controlled by the loaded agent. No input required. Useful for watching trained agents play and observing emergent strategy.
How to Play	A 9-slide interactive tutorial. Slides cover: welcome, the board path, taking turns, capturing, each special square, winning, and strategy tips. Navigate with the arrow buttons.

6. utils.py — The Toolbox

utils.py — Shared helper functions used across the project

Role: reusable utilities that don't belong to any single file

`utils.py` contains small, generic functions that are needed by multiple files. None of these functions knows about Senet specifically — they are general-purpose tools. Keeping them here avoids duplicating code across files and makes them easy to test independently.

Function / Part	What it does — plain English
<code>moving_average(data, window)</code>	Takes a noisy list of numbers (e.g. win results: 0,1,0,0,1,1,0...) and smooths it out by averaging over a rolling window. Without this the training graphs look like random noise; with it the trend becomes visible.
<code>ensure_dir(path)</code>	Creates a folder if it does not already exist (<code>os.makedirs</code> with <code>exist_ok=True</code>). Called before saving models, graphs, or logs so the code never crashes because a folder is missing.
<code>save_training_log(wins, rewards, path)</code>	Writes two lists (win history and reward history) to a CSV file with episode numbers. Useful for post-training analysis or importing into a spreadsheet.
<code>print_progress_bar(current, total)</code>	Prints a text-based progress bar to the terminal during training:  42.0% ep 420/1000. Updates in-place using <code>\r</code> so it does not flood the console.

Function / Part	What it does — plain English
<code>board_to_features(board)</code>	Converts the board tuple into a NumPy array. Not used in the current tabular Q-learning implementation but provides a ready-made interface if a neural network is added later.
<code>count_pieces(board, player)</code>	Counts how many pieces a given player still has on the board. Used internally to check win conditions and can be used for progress-based reward shaping.

□ *board_to_features() is a forward-thinking addition. If you extend this project to use a Deep Q-Network (DQN), this function already provides the feature vector the network needs as input — no changes required elsewhere.*

7. A Single Move — End to End

Here is the complete journey of one move, from your mouse click to the board updating on screen. Following this path through the code is the best way to see how all six files connect.

20. You click a highlighted piece on the board in the Pygame window.
21. `gui.py: _handle_click()` fires, routes to `_handle_cell_click(idx)`.
22. `gui.py: _handle_cell_click()` calls `game.move_piece(idx)`.
23. `game_engine.py: move_piece()` validates the move against `get_valid_moves()`.
24. `game_engine.py`: the board list is updated. Special squares are applied. Win condition is checked.
25. `game_engine.py`: returns (new_state, reward, done, info) to `gui.py`.
26. `gui.py`: updates the status message. If done is True, shows the winner and stops.
27. `gui.py`: calls `game.roll_dice()` and `game.get_valid_moves()` for the next player.
28. If it is the AI's turn, `gui.py` calls `_ai_move()`.
29. `agent.py: choose_action_greedy(state, valid_moves)` looks up Q-values and returns the best move.
30. `gui.py`: calls `game.move_piece(ai_action)` again and the cycle repeats from step 4.

During training the same cycle runs without the GUI, which is why training is thousands of times faster than watching the game in real time.

□ *Notice that `gui.py` never touches `self.board` directly. It always calls `game_engine` functions. This is a design principle called separation of concerns: the GUI knows how to display things; the engine knows the rules. Neither does the other's job.*

8. Quick Reference

Terminal Commands

Function / Part	What it does — plain English
<code>python main.py</code>	Standard start: trains any missing models, then opens the game window.
<code>python main.py --skip-train</code>	Opens the game immediately. Uses whatever models are already in <code>models/</code> .
<code>python main.py --train-only</code>	Trains all three difficulty levels sequentially and exits. No GUI.
<code>python training.py --level easy</code>	Trains only the easy agent (~30 seconds). Saves to <code>models/agent_easy.pkl</code> .
<code>python training.py --level medium</code>	Trains only the medium agent (~2 minutes).
<code>python training.py --level hard</code>	Trains only the hard agent (~10 minutes).

Function / Part	What it does — plain English
<code>python training.py --level all</code>	Trains all three levels in sequence.
<code>python generate_report.py</code>	Regenerates the academic PDF report with embedded graphs.

File Output Reference

Function / Part	What it does — plain English
<code>models/agent_easy.pkl</code>	Saved Q-table for the easy agent. Generated by training.py.
<code>models/agent_medium.pkl</code>	Saved Q-table for the medium agent.
<code>models/agent_hard.pkl</code>	Saved Q-table for the hard agent.
<code>graphs/training_easy.png</code>	Training chart (win rate + reward) for the easy agent.
<code>graphs/training_medium.png</code>	Training chart for the medium agent.
<code>graphs/training_hard.png</code>	Training chart for the hard agent.
<code>logs/log_<level>.csv</code>	Raw per-episode win/reward data. Can be opened in Excel.
<code>report/Senet_QLearning_Report.pdf</code>	The auto-generated academic report.